

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

PROGRAMACIÓN III

INFORME DEL TRABAJO FINAL INTEGRADOR

SISTEMA DE GESTIÓN DE PEDIDOS DE COMIDA

Food Store

Alumno:

Desiderio Silva, Lucas

Profesores:

Alberto Cortez — Cinthia Rigoni

Repositorio en GitHub:

[TPI-Final Programacion-3](#)

Fecha de entrega:

29 de junio de 2026

1. Introducción

El presente documento constituye el informe técnico del Trabajo Final Integrador de la materia Programación III, correspondiente a la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional. El proyecto desarrollado recibe el nombre de Food Store y consiste en un sistema de gestión de pedidos de comida implementado en dos partes complementarias e independientes entre sí: una interfaz web desarrollada con TypeScript y Vite, y un backend de consola construido con Java, JPA, Hibernate y base de datos H2 en archivo.

El sistema permite gestionar de forma integral las cuatro entidades principales del dominio: categorías de productos, productos del catálogo, usuarios del sistema y pedidos con sus líneas de detalle. El frontend opera consumiendo datos desde archivos JSON locales mediante `fetch()`, lo que permite construir y verificar todos los flujos de usuario de manera independiente al backend, para en una iteración futura reemplazar dichas llamadas por peticiones a la API REST correspondiente. El backend, por su parte, expone todas las operaciones a través de un menú de consola navegable, incorporando relaciones JPA, `soft delete`, consultas JPQL personalizadas y gestión transaccional completa.

La elección de este dominio responde a su alta representatividad en el contexto de aplicaciones comerciales reales. Un sistema de gestión de pedidos de comida involucra relaciones entre entidades de distinta cardinalidad, flujos transaccionales con múltiples tablas, control de inventario y separación de roles entre clientes y administradores. Todo ello lo convierte en un escenario ideal para aplicar los conceptos fundamentales de Programación III: persistencia con JPA/Hibernate, patrón Repository, `soft delete`, gestión de transacciones y arquitectura por capas. Asimismo, el frontend permite ejercitar el desarrollo de interfaces web modernas con TypeScript, enrutamiento entre páginas, gestión de estado en `localStorage` y separación de responsabilidades mediante módulos.

1.1 Modelo de Dominio

El modelo de dominio del backend se compone de una clase abstracta `Base`, anotada con `@MappedSuperclass`, que concentra los atributos comunes a todas las entidades: `id` (Long, autogenerated), `eliminado` (boolean, valor por defecto `false`) y `createdAt` (LocalDateTime). Extienden esta clase cinco entidades JPA: `Categoria`, `Producto`, `Usuario`, `Pedido` y `DetallePedido`. Adicionalmente, el sistema define una interfaz `Calculable`, cuyo único método `calcularTotal():void` es implementado por la entidad `Pedido` para calcular su total a partir de la suma de los subtotales de sus líneas de detalle.

La entidad `Categoria` agrupa productos bajo un nombre y una descripción. `Producto` representa un artículo del catálogo con nombre, precio, descripción, stock, imagen y un flag de disponibilidad. `Usuario` modela a las personas que operan el sistema, con nombre, apellido, mail, celular, contraseña y un rol (enumerado `Rol`: ADMIN o USUARIO). `Pedido` es la orden de compra e incorpora fecha, estado (enumerado `Estado`: PENDIENTE, CONFIRMADO, TERMINADO o CANCELADO), total y forma de pago (enumerado `FormaPago`: TARJETA, TRANSFERENCIA o EFECTIVO). Finalmente, `DetallePedido` representa una línea dentro de un pedido, almacenando la cantidad y el subtotal calculado para un producto dado.

El siguiente diagrama UML ilustra el modelo completo con sus relaciones, cardinalidades y enumerados asociados:

Esta separación arquitectónica fue una decisión deliberada dictada por la consigna: permite construir y validar todos los flujos de usuario del frontend de forma completamente independiente al backend. La capa de acceso a datos del frontend está aislada en módulos de utilidades (utils/), de modo que, en una iteración futura, reemplazar un fetch() sobre un archivo JSON por una llamada a la API REST del backend requiere únicamente modificar la URL dentro del módulo correspondiente, sin alterar la lógica de presentación ni los flujos de usuario.

2.2 Estructura de Paquetes del Backend

El proyecto backend es una aplicación Gradle con Java, cuyo código fuente se organiza bajo el paquete raíz com.tp.jpa. La estructura de paquetes refleja una arquitectura en capas con responsabilidades bien delimitadas:

- model/: contiene las entidades JPA (Base, Categoria, Producto, Usuario, Pedido, DetallePedido).
- model/enums/: define los enumerados del dominio (Rol, Estado, FormaPago) y la interfaz Calculable.
- util/: contiene JPAUtil, clase utilitaria que mantiene la instancia singleton de EntityManagerFactory.
- repository/: aloja BaseRepository<T> y los cuatro repositorios específicos (CategoriaRepository, ProductoRepository, UsuarioRepository, PedidoRepository).
- Main.java: clase principal que implementa el menú de consola y coordina todas las operaciones del sistema.

La configuración de la base de datos reside en src/main/resources/META-INF/persistence.xml, donde se define la unidad de persistencia con la URL de conexión H2 en modo archivo (jdbc:h2:file:./data/jpa_db) y la propiedad hbm2ddl.auto=update, que delega en Hibernate la gestión automática del esquema de base de datos.

2.3 Relaciones JPA

El diseño de las relaciones JPA priorizó la direccionalidad mínima necesaria para satisfacer los requerimientos del sistema, evitando referencias bidireccionales innecesarias que incrementan la complejidad del modelo y el riesgo de ciclos de serialización.

La relación Categoria → Producto se modela como agregación unidireccional. Producto declara @ManyToOne con @JoinColumn(name="categoria_id"), lo que genera la clave foránea en la tabla de productos. La categoría puede existir sin productos asociados. La relación Pedido → DetallePedido se modela como composición bidireccional: Pedido declara @OneToMany con cascade=CascadeType.ALL, fetch=FetchType.LAZY y @JoinColumn(name="pedido_id"), mientras que DetallePedido declara @ManyToOne hacia Pedido. El uso de CascadeType.ALL garantiza que la persistencia, actualización y eliminación de un pedido se propague automáticamente a todos sus detalles. FetchType.LAZY evita cargas innecesarias cuando solo se necesitan los datos del pedido sin sus líneas de detalle.

La relación DetallePedido → Producto es unidireccional (@ManyToOne con @JoinColumn(name="producto_id")), dado que el sistema nunca requiere navegar desde un producto hacia los detalles que lo referencian. La relación Usuario → Pedido es unidireccional (@OneToMany con CascadeType.ALL y @JoinColumn(name="usuario_id")), permitiendo obtener todos los pedidos de un usuario sin requerir que el pedido mantenga una referencia explícita al usuario en el modelo de objetos (aunque sí genera la columna de clave foránea en la tabla de pedidos).

2.4 Soft Delete

Todas las bajas del sistema se implementan como bajas lógicas mediante el campo eliminado heredado de Base. Cuando se solicita eliminar una entidad, el sistema establece eliminado=true y persiste el cambio mediante merge(), sin eliminar físicamente el registro de la base de datos. Este enfoque preserva la integridad referencial (un DetallePedido siempre puede referenciar a su Producto, incluso si este fue dado de baja), permite recuperar registros eliminados en caso de error operativo y mantiene el historial completo de datos para auditoría.

El impacto del soft delete sobre las consultas es uniforme en todo el sistema: todas las queries JPQL que retornan entidades activas incluyen la cláusula WHERE e.eliminado = false. El método listarActivos() de BaseRepository<T> implementa esta lógica de manera genérica construyendo la consulta con el nombre simple de la clase (getEntityClass().getSimpleName()), de modo que funciona correctamente con cualquier entidad que extienda Base.

2.5 Gestión de Transacciones

La gestión de transacciones se centraliza en la capa de repositorio siguiendo la regla de que cada método abre su propio EntityManager al inicio y lo cierra en un bloque finally, independientemente de si la operación fue exitosa o no. Las operaciones de escritura (guardar y eliminarLogico) gestionan la transacción explícitamente mediante begin(), commit() y rollback() ante excepción.

La operación más compleja del sistema es el alta de pedido, que involucra validaciones de disponibilidad y stock, reducción de inventario, creación de líneas de detalle y cálculo del total. Esta operación se ejecuta en una única transacción atómica: si cualquier validación falla después de iniciada la transacción, se realiza un rollback completo que garantiza que ningún dato quede modificado parcialmente en la base de datos.

2.6 Consultas JPQL Personalizadas

Los repositorios específicos implementan consultas JPQL tipadas con parámetros nombrados, evitando casteos manuales y garantizando la compatibilidad con el mecanismo de soft delete. A continuación se documentan las cuatro consultas personalizadas del sistema:

buscarProductosPorCategoria (CategoriaRepository):

```
// Retorna los productos activos de una categoría.  
// Navega desde Categoria hacia su colección implícita mediante JOIN.  
String jpql = "SELECT p FROM Categoria c JOIN c.productos p "  
              + "WHERE c.id = :catId AND p.eliminado = false";
```

buscarPorMail (UsuarioRepository):

```
// Busca un usuario activo por su dirección de correo electrónico.  
String jpql = "SELECT u FROM Usuario u "  
              + "WHERE u.mail = :mail AND u.eliminado = false";
```

buscarPedidosPorUsuario (UsuarioRepository):

```
// Retorna los pedidos activos de un usuario navegando la colección
```

```
u.pedidos.  
String jpql = "SELECT p FROM Usuario u JOIN u.pedidos p "  
            + "WHERE u.id = :uid AND p.eliminado = false";
```

buscarPorEstado (PedidoRepository):

```
// Retorna todos los pedidos activos con un estado específico.  
String jpql = "SELECT p FROM Pedido p "  
            + "WHERE p.estado = :estado AND p.eliminado = false";
```

2.7 Frontend — Arquitectura y Decisiones

El frontend fue desarrollado con Vite como herramienta de build, TypeScript como lenguaje principal, HTML5 y CSS3. La aplicación sigue una arquitectura de múltiples páginas (MPA): cada sección del sistema tiene su propio archivo HTML, TypeScript y CSS dentro de una carpeta dedicada bajo `src/pages/`. Todos los entry points se declaran en `vite.config.ts` mediante `rollupOptions.input`, lo que permite a Vite generar bundles optimizados por página.

La capa de acceso a datos está completamente aislada en el directorio `src/utills/`, donde residen los módulos de `fetchs`, `verificarSesion`, `mostrarResultado`, `localStorage` y `navigate` (heredada del modelo básico del parcial número uno). Cada función que obtiene datos realiza un `fetch()` apuntando a los archivos JSON locales en `public/data/`. Esta decisión arquitectónica garantiza que en la iteración siguiente, al conectar el backend JPA, solo sea necesario modificar la URL dentro de estos módulos de utilidades, teniendo también en cuenta el no persistir todo en el `localStorage`.

La autenticación se implementa comparando las credenciales ingresadas contra los datos del archivo `usuarios.json`, con propósitos exclusivamente educativos. Ante un login exitoso, los datos del usuario (sin contraseña) se guardan en `localStorage` y se redirige según el rol: ADMIN al panel de administración, USUARIO al catálogo. Cada página protegida verifica la existencia de la sesión en `localStorage` antes de renderizarse, redirigiendo al login en caso contrario. Las operaciones de escritura (crear, editar, eliminar categorías, productos y cambios de estado de pedidos) se aplican únicamente en memoria: al borrar el `localStorage` de la página, el estado vuelve al original de los archivos JSON, lo cual es intencional en esta iteración.

2.8 Menú de Consola

La clase `Main` implementa un menú principal con seis opciones: Gestionar Categorías, Gestionar Productos, Gestionar Usuarios, Gestionar Pedidos, Reportes y Salir. Cada opción despliega un submenú específico. El orden de uso natural del sistema está documentado en el menú: primero crear categorías, luego productos asociados a una categoría, luego usuarios y finalmente pedidos.

Cada submenú ofrece operaciones de alta, modificación, baja lógica y listado para su entidad correspondiente. El submenú de Pedidos es el más complejo, incluyendo además cambio de estado, consultas por usuario y por estado. El submenú de Reportes agrupa las consultas JPQL personalizadas y el cálculo del total facturado (suma de totales de pedidos en estado TERMINADO, formateado a dos decimales con `Locale.US` para evitar errores de representación de `Double`). La opción Salir cierra correctamente el `EntityManagerFactory` mediante `JPAUtil.close()` antes de terminar la aplicación.

3. Transacciones y Pruebas

3.1 Estrategia Transaccional

La estrategia transaccional del sistema sigue un flujo definido en tres etapas. En primer lugar, todas las validaciones de negocio (campos obligatorios, unicidad de mail, disponibilidad de producto, suficiencia de stock) se realizan fuera del bloque transaccional, para minimizar el tiempo de bloqueo de recursos en la base de datos. En segundo lugar, una vez superadas las validaciones, se abre el EntityManager, se inicia la transacción y se ejecutan todas las operaciones sobre las entidades gestionadas. En tercer lugar, si todas las operaciones son exitosas se realiza el commit; ante cualquier excepción, se invoca el rollback y se muestra un mensaje de error claro al operador.

Esta estrategia garantiza las propiedades ACID en todas las operaciones de escritura: atomicidad mediante commit/rollback explícitos, consistencia mediante validaciones previas y constraints de la base de datos, aislamiento al nivel por defecto de H2 y durabilidad al persistir en archivo (jdbc:h2:file). El alta de pedido es el caso más representativo: la creación del pedido, la inserción de sus detalles y la reducción del stock de cada producto ocurren dentro de una única transacción, de modo que si cualquiera de estas operaciones falla, ninguna modificación queda persistida.

3.2 Casos de Prueba Manuales

Categorías:

- Alta válida: nombre y descripción ingresados → categoría creada, ID generado mostrado correctamente.
- Alta inválida: nombre vacío → error informado, operación cancelada sin persistir.
- Modificación: campo vacío → valor anterior conservado; nuevo valor ingresado → cambio persistido.
- Baja lógica: eliminado=true, categoría no aparece en listados activos; confirmación con nombre mostrada.

Productos:

- Alta sin categorías activas → error informado, operación cancelada.
- Alta con precio ≤ 0 o stock < 0 → validación detectada, no se persiste.
- Alta válida → producto creado con categoría asignada, ID generado mostrado.
- Baja lógica → producto no aparece en catálogo ni en selección de productos al crear pedido.

Usuarios:

- Alta con mail duplicado → buscarPorMail retorna Optional no vacío, error informado, sin persistencia.
- Alta válida → usuario creado con rol asignado, ID generado mostrado.
- Buscar por mail → Optional.of(usuario) mostrado sin contraseña; mail inexistente → Optional.empty() informado.
- Baja lógica → usuario no aparece en listados, sus pedidos permanecen en el sistema.

Pedidos:

- Alta con producto no disponible → validación rechazada antes de la transacción.
- Alta con stock insuficiente → stock disponible mostrado, operación cancelada.
- Alta válida → pedido creado, detalles persistidos por cascade, stock reducido, total calculado mostrado.
- Cambio de estado → nuevo estado persistido, confirmación mostrada.

- Baja lógica → pedido no aparece en listados, stock de productos no restaurado, detalles permanecen en BD.
- Reporte total facturado: suma de totales de pedidos TERMINADO mostrada con formato \$XX.XX.

4. Herramientas y Tecnologías

4.1 Stack Tecnológico

El backend fue desarrollado en Java 21 (LTS) utilizando Gradle como herramienta de build. La capa de persistencia emplea JPA como especificación y Hibernate 6 como proveedor ORM, con base de datos H2 en modo archivo como motor de almacenamiento. El frontend fue desarrollado con Vite 7 como bundler y servidor de desarrollo, TypeScript como lenguaje, HTML5 y CSS3 para la interfaz. El control de versiones se gestionó con Git a través de GitHub.

4.2 Decisiones Técnicas

La elección de JPA/Hibernate sobre JDBC puro responde a múltiples ventajas para este dominio. JPA elimina la necesidad de escribir SQL para operaciones CRUD estándar, gestiona automáticamente el esquema mediante `hbm2ddl.auto=update`, y permite expresar las relaciones entre entidades de forma declarativa mediante anotaciones. Esto resulta en un código más limpio, mantenible y extensible, especialmente cuando el modelo de dominio involucra múltiples entidades relacionadas como en Food Store. Las consultas personalizadas se expresan en JPQL, que opera sobre el modelo de objetos en lugar de sobre tablas, resultando en queries independientes del motor de base de datos subyacente.

La elección de H2 en modo archivo (`jdbc:h2:file:./data/jpa_db`) sobre una base de datos de servidor como MySQL o PostgreSQL responde al contexto académico del proyecto: H2 no requiere instalación ni configuración de servidor, la base de datos es portable (es un archivo en el filesystem) y Hibernate gestiona automáticamente su creación y evolución del esquema. En un contexto productivo, la migración a otro motor relacional requeriría únicamente cambiar la URL de conexión y el driver en `persistence.xml`.

En el frontend, la elección de Vite sobre Create React App u otras alternativas responde a su velocidad de arranque, soporte nativo para TypeScript sin configuración adicional, y su modelo de build basado en Rollup que genera bundles optimizados por página. La arquitectura MPA (múltiples HTMLs) fue preferida sobre una SPA (React Router) dado que simplifica el enrutamiento entre secciones para esta iteración sin backend.

4.3 Uso de Inteligencia Artificial

Durante el desarrollo del proyecto se utilizó Claude (Anthropic) como herramienta de asistencia. Sus usos principales incluyeron la revisión de código y detección de bugs, la generación de fragmentos boilerplate (asistencia en configuración de `persistence.xml` y estructura inicial de `vite.config.ts`), sugerencias de optimización en consultas JPQL, asistencia en la redacción de la documentación técnica, asistencia en la elección de colores empleados en el frontend, actuando como consultor UX/UI. Todas las decisiones de diseño y arquitectura fueron tomadas por el alumno; la IA fue utilizada de manera ética y académica, operando exclusivamente como herramienta de soporte y verificación, no como generadora de la solución.

4.4 Patrones de Diseño Implementados

El patrón más relevante del sistema es el Repository Pattern, implementado mediante la clase abstracta genérica `BaseRepository<T>`. Este patrón abstrae la lógica de acceso a datos detrás de una interfaz uniforme de operaciones CRUD, evitando la duplicación de código de persistencia en cada repositorio específico. Cada repositorio concreto hereda estas operaciones e incorpora únicamente las consultas específicas de su dominio.

El patrón Template Method se materializa en `listarActivos()`, que define el algoritmo genérico de consulta con filtro de soft delete, parametrizando dinámicamente el nombre de la entidad mediante `getEntityClass().getSimpleName()`. El patrón Singleton es aplicado por `JPAUtil` para garantizar que exista una única instancia de `EntityManagerFactory` en toda la aplicación, dado que su creación es costosa y debe realizarse una sola vez. En el frontend, el patrón Module encapsula la capa de fetch en utilidades independientes, aplicando el principio de responsabilidad única y facilitando la futura sustitución de la fuente de datos.

5. Análisis de Resultados

5.1 Cumplimiento de Objetivos

El sistema desarrollado cumple con la totalidad de los requerimientos funcionales y técnicos establecidos en la consigna del Trabajo Final Integrador. El backend implementa el modelo de dominio completo con sus cinco entidades JPA, cuatro repositorios con CRUD genérico y consultas JPQL personalizadas, soft delete en todas las entidades, gestión transaccional atómica en el alta de pedido con reducción de inventario, y un menú de consola completamente navegable con los cinco submenús requeridos. El frontend implementa los diez flujos de usuario definidos: login y registro con roles, catálogo con filtros y búsqueda en tiempo real, detalle de producto con validación de stock, carrito persistido en `localStorage`, checkout con forma de pago, historial de pedidos del usuario, y panel de administración completo con dashboard, CRUD de categorías, CRUD de productos y gestión de pedidos con cambio de estado.

5.2 Aspectos Destacados

Entre los aspectos más destacados de la implementación se encuentra la robustez transaccional del alta de pedido: la garantía de atomicidad mediante una única transacción que abarca la creación del pedido, sus detalles y la reducción de stock de múltiples productos evita inconsistencias en la base de datos ante cualquier fallo en el proceso. La separación arquitectónica entre la capa de fetch y la lógica de presentación en el frontend es otro punto destacable, ya que hace que la futura integración con el backend JPA sean cambios mínimos y localizados. La implementación del patrón Repository genérico elimina toda duplicación en el código de persistencia y facilita la incorporación de nuevas entidades al sistema sin modificar las existentes.

5.3 Limitaciones y Mejoras Futuras

La principal limitación del sistema en su estado actual es la ausencia de integración entre el frontend y el backend: el frontend opera sobre datos JSON estáticos y las operaciones de escritura no son persistentes entre recargas de página. Esta limitación es intencional en esta primera iteración, siendo la conexión REST el objetivo natural de la siguiente etapa de desarrollo. En el frontend, la autenticación en texto plano contra el JSON y la gestión de sesión mediante `localStorage` son aceptables en un contexto académico pero no para un entorno productivo.

Las mejoras futuras más relevantes incluyen el desarrollo de una API REST con Spring Boot sobre el backend JPA existente, la integración del frontend mediante reemplazo de las URLs de fetch, la implementación de autenticación real con tokens JWT, la externalización de la configuración de la base de datos mediante variables de entorno, la adición de un pool de conexiones para escenarios de

mayor concurrencia, y la migración de H2 a un motor de base de datos productivo como PostgreSQL para el ambiente de despliegue.

6. Conclusiones

El desarrollo del sistema Food Store permitió aplicar de forma integral los conceptos centrales de Programación III en un dominio con complejidad representativa del mundo real. La implementación del patrón Repository con generics, la gestión de relaciones JPA con distintas cardinalidades y direccionalidades, el diseño del soft delete como decisión transversal al modelo, y la coordinación de transacciones atómicas en operaciones multi-entidad constituyen los aprendizajes técnicos más significativos del proyecto.

La arquitectura en dos partes independientes demostró ser una decisión acertada: permitió avanzar en paralelo sobre el frontend y el backend sin dependencias bloqueantes entre ambos, y preparó el terreno para la integración futura mediante una capa de fetch claramente delimitada. La elección de JPA/Hibernate sobre JDBC puro resultó en un código más expresivo y mantenible, donde las relaciones entre entidades se leen directamente en el modelo de objetos sin necesidad de interpretar joins SQL.

Más allá de los aspectos técnicos, el proyecto representó una oportunidad de tomar decisiones de diseño con consecuencias reales: la elección de relaciones unidireccionales versus bidireccionales, el alcance del cascade, la granularidad de las transacciones y la separación de responsabilidades entre capas son decisiones que tienen impacto directo en la calidad, mantenibilidad y extensibilidad del sistema. Las limitaciones identificadas no son defectos, sino decisiones conscientes y documentadas propias del contexto académico e iterativo en el que fue desarrollado el trabajo.

7. Referencias

Documentación oficial:

Eclipse Foundation. (2024). *Jakarta Persistence 3.1 Specification*.
<https://jakarta.ee/specifications/persistence/>

Hibernate. (2024). *Hibernate ORM 6 User Guide*.
https://docs.jboss.org/hibernate/orm/6.0/userguide/html_single/

H2 Database Engine. (2024). *H2 Database Engine Documentation*.
<https://h2database.com/html/main.html>

Oracle. (2024). *Java Platform, Standard Edition 21 Documentation*.
<https://docs.oracle.com/en/java/javase/21/>

Vite. (2024). *Vite Documentation*. <https://vite.dev/>

Microsoft. (2024). *TypeScript Documentation*. <https://www.typescriptlang.org/docs/>

Herramientas utilizadas:

- Claude (Anthropic): consultor de dudas, generación de documentación y sugerencias de optimización.
- IntelliJ IDEA: entorno de desarrollo integrado para el backend Java.
- Visual Studio Code: editor principal para el desarrollo del frontend TypeScript.
- GitHub: control de versiones y repositorio del proyecto.